

Clase Language

SPECIFICATION & LANGUAGE REFERENCE MANUAL

An Object-Oriented Programming (OOP) language blending new **classes** with syntactical **elegance** (both designated by *Clase* in Spanish). Designed specifically to streamline AI-driven coding by unifying program flow in a natural, left-to-right writing direction.

1. The Basics: Syntax & Variables

Clase is engineered with a primary goal: to make AI coding intuitive, seamless, and aligned with standard human handwriting. It enforces a unifying rule where assignments, variable and function declarations, function calls, and input/output operations flow strictly from left to right.

Core Syntactical Paradigm

Traditional languages retain assembly-era conventions where operators appear on the left and operands on the right (e.g., `MOV R1, R2` or `x = 5`). Clase breaks away from this legacy model to establish a unified left-to-right evaluation chain.

Key Concepts

- **New Assignment Operator:** Instead of `variable = value`, Clase utilizes `value > variable`. This allows natural left-to-right stream processing (e.g., `a + b > a`). Note: In conditional expressions, the relational operator `>` ("greater than") retains its standard evaluation logic.
- **Renamed Control Flow Keywords:**

Standard Keyword	Clase Equivalent	Purpose
<code>if</code>	<code>when</code>	Conditional execution block
<code>else</code>	<code>other</code>	Alternative execution branch
<code>while</code>	<code>repeat</code>	Conditional loop block
<code>for</code>	<code>iterate</code>	Counter-based iteration loop
<code>function</code>	<code>fun</code>	Function block definition
<code>return</code>	<code>back</code>	Return value from routine

- **Semitagged Keywords:** Control flow and structural blocks are opened and closed using the pipe symbol (`|`), mimicking markup structures. Blocks are explicit and self-closing: `repeat | ... |repeat` , `fun | ... |fun` , and `when | ... |when` . This explicit scoping simplifies error detection and syntax parsing.
- **Comments:** Single-line comments begin with double slashes (`//`).

Data Types & Operators

Clase supports standard primitive types and arithmetic operators (`+` , `-` , `*` , `/` , `%`):

`int` Integer values `float` Floating-point decimals `char` Character literals `string` Character vectors
`bool` Boolean (`true` / `false`)

Variable Declaration & Initialization

- **Declaration only:** `int i`
- **Declaration with initialization:** `int i(0)` , `float(3.14)` , `char c("a")` , `bool l(true)` , `string s("abcde")`
- **Multiple declarations:** Variables of the same type are declared using whitespace separation: `int a b`

2. Program Structure & Functions

Clase programs are organized into modules, functions, and a main entry point.

Modules & Function Syntax

A module is defined within a `module |` block:

```
module | Euclidean
    // Module contents
|module
```

Function declarations follow the left-to-right parameter-to-return pattern:

- **Syntax:** `fun | <input_parameters> > <function_name> > <return_type>`
- **Example:** `fun | int a int b > Euclid > int` (Declares function `Euclid` taking two integers and returning an integer).
- **Invocation:** `a b > f`

The Main Function & Input/Output Chaining

The program entry point is declared as `fun | string args | | > main > int` .

Example 1: The Euclidean Algorithm

Demonstration of left-to-right I/O streaming, functions, and loop constructs.

```

module| Euclidean                                // module definition

    fun| int a int b > Euclid > int // function declaration

        repeat| b != 0
            when| a > b
                a - b > a
            other|
                b - a > b
            |other
        |when
    |repeat

    back| a |back // return value (scalar, not vector)

|fun

fun| string args| | > main > int // entry point

    int a b

    in > a b > Euclid > out

    // Combined input, function execution, and output chaining
    // Seamless stream processing syntax

|fun

|module

```

The expression `in > a b > Euclid > out` seamlessly performs five sequential operations in a single line: (1) Accepts standard input, (2) Stores values into `a` and `b`, (3) Passes parameters into `Euclid`, (4) Captures the evaluation result, and (5) Directs the result to standard output.

3. Control Flow & Vectors

Conditionals & Loops

Control flow relies on explicit block wrappers. Below are standard control structures:

```
// Conditional Branching
when| a > b
    a - b > a
other|
    b - a > b
|other
|when

// Repeat Loop
repeat| b != 0
    // Loop body
|repeat
```

Vectors & Iteration

Clase uses contiguous vectors instead of standard arrays for performance optimization, using pipe brackets `| |` for indexing and bounds definition.

- **Declaration:** `int a|n|` (declares an integer vector with `n` elements).
- **Initialization:** `int a|4|(|1, 2, 3, 4|)`.
- **Access:** `a|i|` (0-based indexing).
- **Iteration Syntax:** `iterate|int i(0)++ < n` (declares iterator `i=0`, increments by 1 each step, runs while `i < n`).

```
iterate| int k(0)++ < n          // fill vector with true
    true > primes|k|
|iterate
```

4. Advanced Example: Sieve of Eratosthenes

The following module implements prime number generation using vector manipulation, nested loops, and formatted console output.

```

module| primes

  fun| int n > Eratosthenes > int          // function declaration

    bool primes | n |                    // vector declaration
    int l(0), i(0), index_square(3)
    int first, last, factor

    iterate| int k(0)++ < n              // fill vector with true
      true > primes[k]
    |iterate

    repeat| index_square < n
      when| primes[i]
        0 + index_square > first
        0 + n > last
        i + i + 3 > factor
        false > primes[first]

        repeat| last - first > factor
          first + factor > first
          false > primes[first]
        |repeat
      |when
        i + 1 > i
        2 * i * (i + 3) + 3 > index_square
    |repeat

    ' 2' > out // output base prime

    iterate| i(0)++ < n                  // print remaining primes
      when| primes[i]
        when| 2 * i + 3 > n
          break
        |when

          ' ' 2 * i + 3 > out
          l + 1 > l

          when| l % 10 == 0
            '

    ' > out

    |when
    |when
    |iterate

    '

  number: ' , l > out

|fun

fun| string args| | > main > int          // main entry point
  1000 > Eratosthenes                    // function call with literal parameter
|fun

```

```
|module
```

5. Intermodular Connectivity

Modules interact through `intermodular|` definitions placed prior to the module implementation. Intermodulars manage library linking and export function signatures across source files (which use the `.Clase` extension).

```
intermodular|  
  
  link| in2out.C clase |link      // link standard I/O library  
  link| module2.C clase |link     // link external module  
  
  fun| int a int b > Euclid > int |fun // exported function header  
  
|intermodular  
  
module| Euclidean  
  
  fun| int a int b > Euclid > int  
    // Implementation  
  |fun  
  
|module
```

6. File System Operations

File handling is managed via the `file` descriptor type: `file f(name, type, options)`.

- **Types:** `txt` (text), `bin` (binary), `hex` (hexadecimal).
- **Options:** `r` (read), `w` (write), `+` (append/update).

```
file f("Readme.md" txt wr+) // declare text file handle  
string s                    // buffer string  
  
f.open  
repeat| s  
  s > f // write string stream to file  
|repeat  
f.close  
  
f.open  
repeat| not f.eof // read until end of file  
  f > s // read file stream into string  
|repeat  
f.close
```

7. Compile-Time Semantics & Generics

Clase eliminates preprocessors and template macro expansions by introducing `comp| ... |comp` blocks. Code inside compilation blocks is executed at build time to facilitate generic type and operator binding (e.g., `&T` and `&Op` parameters).

```
module| generics

comp|                                     // compile-time execution block
    fun| &T a &Op &T b > Result > &T      // generic type T and operator Op

        back| a Op b |back              // return computed expression

    |fun
|comp

fun| string args| | > main > int

    float a_fl(1.41) b_fl(1.73)          // float initializations
    int a_int(1) b_int(2)                // integer initializations

    a_fl + b_fl > Result > out            // resolves to float addition -> 3.14
    a_int + b_int > Result > out          // resolves to integer addition -> 3

    a_fl * b_fl > Result > out            // resolves to float multiplication -> 2.44
    a_int * b_int > Result > out          // resolves to integer multiplication -> 2

|fun

|module
```

8. Object-Oriented Programming: Compile-Time Classes

Clase supports object-oriented paradigms with compile-time class evaluation. Classes define member attributes and functions encapsulated within `class| ... |class` constructs.

```

module| Euclidean

comp|          // compile-time class evaluation

  class| GCD

    int a int b    // member attributes

    fun| int a int b > Euclid > int    // member function
      repeat| b != 0
        when| a > b
          a - b > a
        other|
          b - a > b
        |other
      |when
      |repeat

      back| a |back
    |fun

  |class

|comp

fun| string args| | > main > int

  GCD N    // instantiate object N of class GCD

  in > N.a N.b > N.Euclid > out    // chained stream assignment, execution, & output

|fun

|module

```

9. Concurrency & Memory Management

Multitex Concurrency

Concurrency is natively handled via the **Multitex** task structure, dispatching tasks asynchronously across execution threads.

```

Multitex tasks||          // declare task vector

a b > function1 > tasks|0|    // bind functions to task slots
i n > function2 > tasks|1|

int i(0)
repeat| !eot                // execution loop until end-of-time flag

    tasks|i++|.concurr      // execute tasks concurrently

|repeat

```

Memory Management

Clase incorporates an automated runtime Garbage Collector (GC), handling memory allocation and deallocation dynamically without manual pointer tracking.

10. Language Origins

The design of the **Clase** language draws structural and syntactical inspiration from two foundational computing languages:

- **C++:** Inspired variable declaration patterns (e.g., `int i`) and stream-based input/output paradigms (`cin >> a >> b` / `cout << Euclid(a, b)` mapped to `in > a b > Euclid > out`).
- **Common Lisp:** Derived explicit, self-closing pipe delimiters for code blocks (e.g., `|fun` , `|when`), modeled after Common Lisp multi-line comment syntaxes (`#| ... |#`).